



PROJET TUTORÉ

RÉSEAUX SOCIAUX

Étude et expérimentation d'un service NoSQL

Résumé :

Dans le document qui suit, vous pourrez étudier et comprendre l'ensemble de notre cas d'étude sur les réseaux sociaux avec Neo4j.

Auteurs :

Valentin PERROUX
Sefer TASDEMIR
Feyza TINASTEPE

SOMMAIRE

1. Présentation du groupe et du contexte choisi	3
1) Membres et rôles du groupe	3
2) Cas d'étude	3
3) Contexte.....	3
4) Justification du choix du cas d'étude.....	4
2. Étude avant-projet : analyse et comparaison	5
1) Neo4j	5
2) MongoDB.....	5
3) SPARQL	6
4) Tableau comparatif résumé.	6
5) Justification du choix NoSQL : Neo4j	7
3. Expérimentation de Neo4j	8
1) Environnement et configurations utilisé	8
2) Problèmes rencontrés.....	9
3) Captures d'écran.....	9
4. Conception d'un jeu de données adapté	11
1) Description du type de données à stocker.....	11
2) Justification du modèle de données choisi	13
3) Aperçu du schéma et structure JSON	14
4) Procédure de génération	15
5. Scénarios de test réalisés	17
1) Insertion de données	17
2) Requêtes simples et avancées.....	18
3) Agrégations	20
4) Mise à jour.....	22
5) Suppression.....	23
6) Analyse critique	24

6. Analyse critique et retour d'expérience	26
1) <i>Points forts et points faible de Neo4j</i>	<i>26</i>
2) <i>Difficultés et amélioration possible du projet</i>	<i>26</i>
3) <i>Répartition réelle du travail.....</i>	<i>27</i>

1. Présentation du groupe et du contexte choisi

1) Membres et rôles du groupe

Nous sommes un groupe composé de trois membres :

- **Valentin Perroux** : responsable de la recherche et de la collecte des informations relatives au cas d'étude.
- **Sefer Tasdemir** : responsable de l'analyse comparative des solutions NoSQL et coconcepteur du modèle.
- **Feyza Tinastepe** : responsable de la conception et la structuration du jeu de données à utiliser pour les tests, s'assurer de la cohérence et de la *scalabilité* du modèle de données.

Chaque membre joue un rôle distinct dans l'analyse du cas d'étude en fonction de ses compétences.

2) Cas d'étude

Le cas d'étude que nous avons choisi est l'analyse et l'implémentation d'un service NoSQL pour gérer les données liées à un **réseau social**. Le but est de simuler un environnement typique d'un réseau social, où les utilisateurs peuvent partager des publications, interagir avec d'autres utilisateurs, commenter des publications, etc.

Les défis pour un service NoSQL dans ce contexte sont nombreux, en particulier en termes de scalabilité et de performance des requêtes sur de grandes quantités de données.

3) Contexte

Les réseaux sociaux sont aujourd'hui au cœur de la transformation numérique de nombreuses entreprises et sont essentiels pour la communication et les interactions en ligne. La gestion des données dans ce domaine pose plusieurs défis, le **volume**, la **diversité** et la **rapidité** de changement des informations générées par les utilisateurs.

Dans ce contexte, les bases de données NoSQL sont de plus en plus utilisées pour leur flexibilité et leur capacité à gérer des données massives et hétérogènes. Les réseaux sociaux génèrent des quantités énormes de données structurées et non structurées, comme des publications, des photos, des vidéos, des likes, des commentaires, des messages privés. Le service NoSQL doit être capable de stocker ces informations de manière efficace et de fournir des performances optimales lors des accès à ces données.

L'objectif de ce projet nous insiste donc à tester et justifier l'utilisation d'un service NoSQL qui répond à ces besoins, dans le cadre d'un réseau social simulé.

4) Justification du choix du cas d'étude

Nous avons choisi d'étudier le cas du réseau social responsable car il représente à la fois un défi technique et une réflexion intéressante. Sur le plan technique, un réseau social repose sur des relations riches entre utilisateurs et sur des contenus variés. Cela nous donne l'occasion de comparer concrètement plusieurs bases NoSQL et d'évaluer leur pertinence face à ces contraintes, notamment pour les parcours relationnels et les agrégations. Sur un deuxième plan, l'idée d'un réseau social sans dark patterns, respectueux de la vie privée et sobre en ressources est motivante car elle va à l'encontre des modèles actuels et oblige à penser des choix architecturaux responsables. Enfin, ce cas nous intéresse personnellement parce que les réseaux sociaux font partie de notre quotidien, travailler sur un sujet familier nous permet de mieux relier la théorie apprise à une application concrète et parlante.

2. Étude avant-projet : analyse et comparaison

Dans le cadre de ce projet, nous avons confronté trois familles NoSQL adaptées aux usages d'un réseau social responsable : **Neo4j** (graphe), **MongoDB** (document) et un entrepôt RDF en **SPARQL**. L'objectif est d'évaluer, dans notre contexte précis, leur capacité à modéliser des relations, à supporter des agrégations et à offrir une visualisation compréhensible pour la restitution. Dans les paragraphes suivants, nous allons justifier notre choix final.

1) Neo4j

Neo4j modélise nativement les interactions sociales sous forme de nœuds et de relations et se requête en Cypher. Ce choix s'adaptera parfaitement à notre besoin de parcours relationnels, parmi eux retrouver le fil d'actualité d'un utilisateur via les comptes suivis, proposer des recommandations par chemins de longueur deux ou trois (amis d'amis), identifier des communautés (nœuds hubs), le tout dans le but de visualiser ces structures. Dans ce contexte, Neo4j facilite la formulation de questions métier complexes avec très peu de code et fournit une visualisation native simple d'utilisation. En contrepartie, nous ne pouvons pas stocker les médias (les photos par exemple) dans Neo4j. Ces données restent en « object storage » avec seulement l'URL et les métadonnées dans la base, il faut également anticiper le *tuning* mémoire et les index lorsque le volume s'accroît ou que le rythme d'écriture devient soutenu.

2) MongoDB

MongoDB repose sur un modèle de documents JSON/BSON flexible et sur une « Aggregation Pipeline » orienté pour les statistiques. Appliqué à notre cas, cela rend naturel le stockage des posts (texte, date, référence) et des commentaires, qui peuvent être imbriqués ou référencés selon la profondeur et les volumes attendus. Les agrégations temporelles courantes comme les classements des posts les plus likés sur les sept derniers jours, activité par pays, se traduisent efficacement en pipelines, et l'écosystème facilite la création de vues produit. La limite apparaît lorsqu'on multiplie les parcours relationnels profonds : les requêtes de type « amis d'amis » deviennent vite coûteuses et invitent à des duplications qui alourdissent la cohérence de la base. MongoDB excelle donc pour le contenu et les agrégations sur les propriétés, mais n'est pas optimal dès que la relation elle-même devient le cœur de la valeur.

3) SPARQL

L'option SPARQL s'appuie sur un modèle RDF où chaque information est un triplet (sujet, prédicat, objet), avec la possibilité d'aligner les données sur des vocabulaires publics (FOAF, SIOC, schema.org) et d'exploiter des requêtes déclaratives, y compris des propriétés de chemin. Transposé à notre réseau social, cela permet de représenter proprement les relations de suivi, les posts, les commentaires et même les likes. Le tout en ouvrant la porte à l'interopérabilité et à la publication en open data. En pratique, cette approche introduit toutefois des frictions pour notre objectif pédagogique, la modélisation demande une ingénierie sémantique plus lourde (les alignements par exemple), les écritures fréquentes (likes/commentaires) peuvent devenir verbeuses, et même si SPARQL couvre bien les agrégations, l'ergonomie pour des parcours sociaux performants et répétitifs dépend fortement du moteur sous-jacent. La visualisation orientée « graphe social » est aussi moins « clé en main » que sur un moteur graphes attribués.

4) Tableau comparatif résumé.

Critère	Neo4j	MongoDB	SPARQL
Description technique	Base graphe native (property graph), requêtes Cypher	Document store JSON/BSON, Aggregation Pipeline	Triplestore RDF, requêtes SPARQL
Cas d'usage typique (projet)	Parcours sociaux, reco « amis d'amis », communautés	Stockage posts et commentaires, métriques temporelles, dashboards produit	Modèle sémantique aligné (FOAF/schema.org),
Avantages clés	Relations et chemins courts très efficaces, visualisation graphe intégrée	Agrégations/classements très performants itération rapide sur le contenu	Interopérabilité, sens des relations explicite, vocabulaire standard
Limites clés	Médias hors base mémoire/index agrégations moins naturelles	Parcours relationnels profonds coûteux besoin de dénormaliser/cacher	Modélisation lourde écritures verbeuses
Agrégations	Correctes	Excellentes	Correctes

5) Justification du choix NoSQL : Neo4j

Comparativement, si l'on met en regard nos besoins prioritaires comme les relations entre utilisateurs, les scénarios de recommandation, les agrégations et visualisations pour comprendre les communautés, le tout en évitant les darks patterns. C'est Neo4j qui offre l'adéquation la plus directe, car le graphe attribué colle exactement au problème à résoudre et aux démonstrations attendues en séance. MongoDB demeure un très bon choix pour la couche « contenu » et les agrégations documentaires, tandis que SPARQL devient pertinent seulement si l'objectif est l'interopérabilité sémantique et la publication de données liées.

En conséquence, nous retenons Neo4j comme moteur principal. Ce choix maximise la lisibilité des requêtes, la capacité de démonstration visuelle pendant la restitution, et l'alignement avec la promesse d'un réseau social responsable où les mécanismes de recommandation peuvent être compris, audités et justifiés.

3. Expérimentation de Neo4j

1) Environnement et configurations utilisé

Dans notre environnement de développement, nous allons utiliser Neo4j Aura (formule gratuite) en tant que base de données NoSQL. Pour faciliter le travail collaboratif nous allons créer un répertoire GitHub, bien sûr les identifiants et mot de passe de l'instance Neo4j sont conservés dans un fichier environnement (ignoré dans les commit). Pour simplifier le lancement du projet, nous allons créer un environnement venv et un fichier requirement.txt, qui permettent une exécution plus simple du projet. Les requêtes seront exécutées dans le fichier « *queries.cypher* » ou sur le site Neo4j, enfin, le fichier « *generator.py* » permet la création des nœud et des relations et « *import.py* » permettra l'import des données générées vers la base.

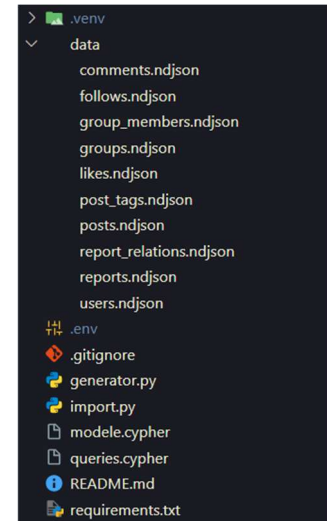


Figure 1 : arborescence du projet

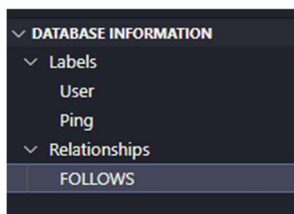


Figure 2 : modèle d'exemple

Sur VS Code nous allons utiliser

[l'extension officiel de Neo4j](#) qui permet également de se connecter à la base, mais aussi de vérifier facilement les nœuds et les relations créés. L'environnement est en Python 3.12.1 avec les dépendances Neo4j 6.0.0, python-dotenv 1.1.1, faker 37.8.0 et pandas 2.3.3.

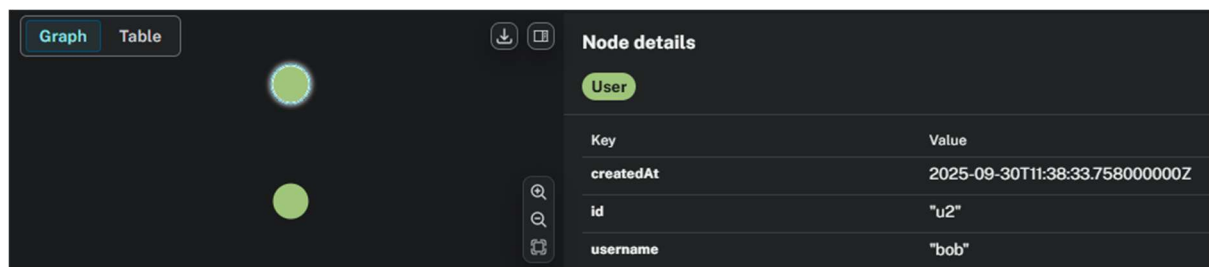


Figure 3 : résultat d'une requête vu grâce à l'extension Neo4j (exemple)

2) Problèmes rencontrés

Authentification : Neo.ClientError.Security.Unauthorized. Il fallait régénérer le mot de passe dans AuraDB, vérifier les variables .env et le launch.json qui pointe bien sûr .env.

A part ce problème, aucune autre erreur fut rencontrée.

3) Captures d'écran

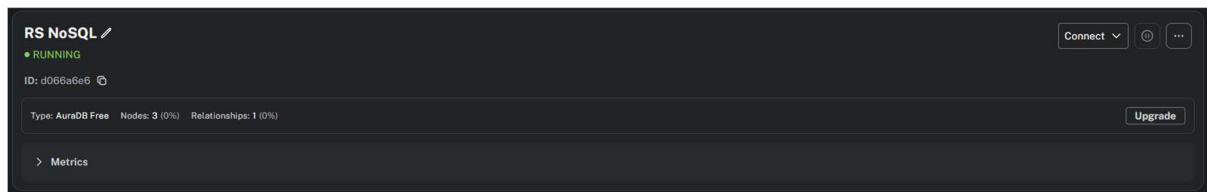


Figure 4 : création de l'instance depuis Neo4j

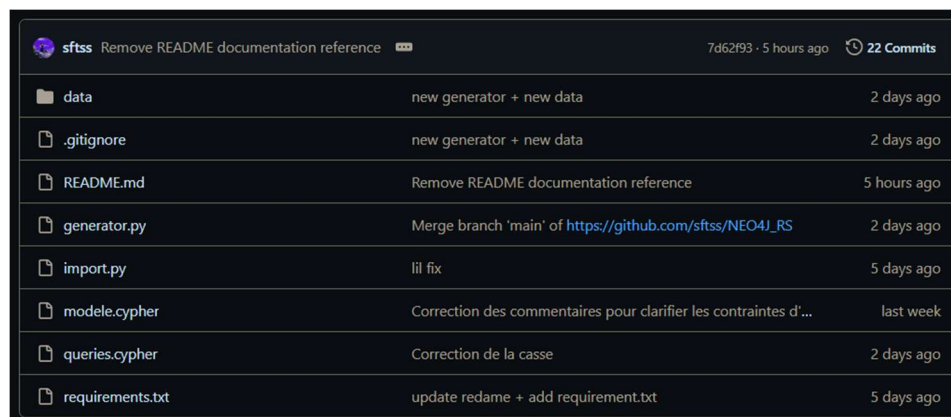


Figure 5 : création du répertoire GitHub

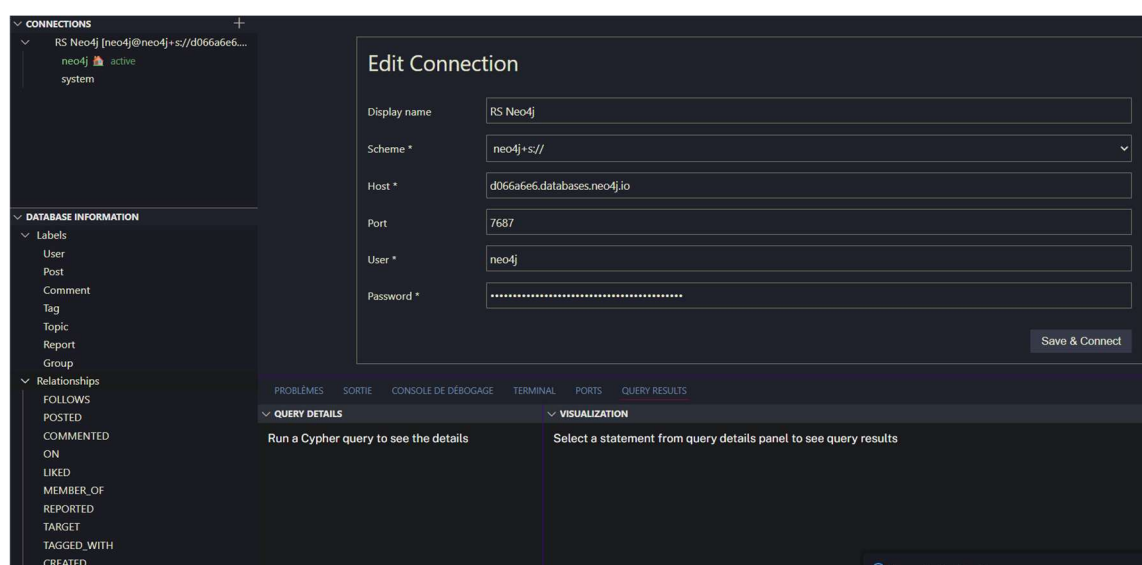


Figure 6 : connexion du graphe sur l'extension de Neo4j depuis Visual Studio Code

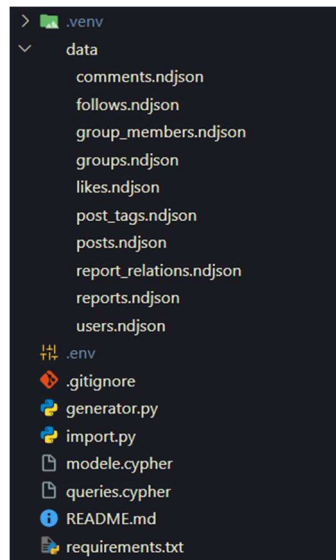
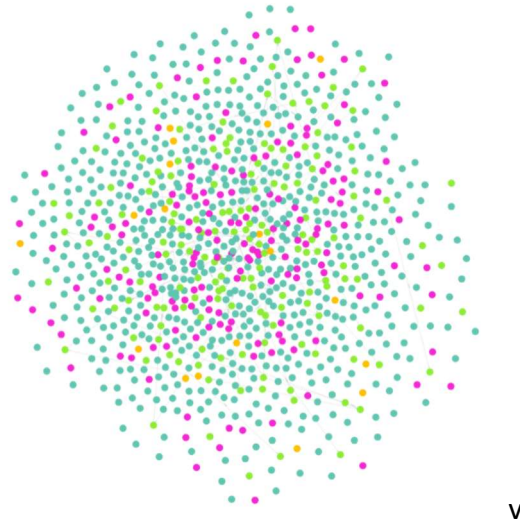


Figure 7 : développement des scripts et génération des données



V

Figure 8 : visualisation des données depuis Neo4j Aura

4. Conception d'un jeu de données adapté

1) Description du type de données à stocker

Le jeu de données simule un réseau social « responsable » et couvre les objets métier attendus : utilisateurs, relations d'abonnement, publications, commentaires, modération, ainsi que la structuration communautaire. Chaque enregistrement est stocké en NDJSON avec des dates afin de faciliter les calculs avec les filtres temporels.

Utilisateurs (User) : chaque compte possède un identifiant, un username, un email, un mot de passe hashé un name généré en français, un niveau de privacy (public ou private) et un createdAt. Afin de pouvoir étudier les effets de proximité sociale et les affinités, chaque utilisateur est rattaché à une **communauté thématique** (tech, sport, gaming, cuisine, voyage) qui influence ses liens et ses contenus.

```
{"id": "u_00000", "username": "laurentnicole", "email":  
"laurentnicole@example.com", "passwordHash":  
"$2b$04$QKQEt8OuIy7EqU4VZpeliubjy7/8GgrQCeyzo92bz1jNcxLNGJ0SW",  
"name": "Gérard-Frédéric Joly", "privacy": "private", "createdAt":  
"2025-05-28T13:56:56", "community": "gaming"}
```

Abonnement (FOLLOWS) : les relations (followerId => followedId) portent un since temporel. Le graphe est **mixte intra/inter-communautés** et présente des **hubs** populaires (attachement préférentiel), ce qui reflète le comportement observé sur de vrais réseaux.

```
{"followerId": "u_00042", "followedId": "u_01234", "since": "2025-05-  
11T17:03:49"}
```

Publications (Post) : une publication contient un text, une visibility (public, followers_only, private), éventuellement une mediaUrl pointant vers une image/vidéo/gif (générés aléatoirement), un createdAt, un champ thématique topic dérivé de la communauté de l'auteur, ainsi que les compteurs likeCount et commentCount) recalculés en fin de génération.

```
{"id": "p_000123", "authorId": "u_01234", "content": "Phrase réaliste en  
fr_FR...", "visibility": "public", "mediaUrl": "https://cdn.socialnet.com/  
media/9k3u1e5m0a2b.jpg", "createdAt": "2025-02-  
06T12:40:55", "topic": "topic_tech", "likeCount": 73, "commentCount": 11}
```

Tags des publications (TAGGED_WITH) : les posts sont annotés par **1 à 5 tags** au format {postId, tagName}. Environ 80 % des tags proviennent du vocabulaire de la **communauté** de l'auteur (python pour *tech*) et 20 % proviennent d'un fond générique (lifestyle, nature, ...).

```
{"postId":"p_000123","tagName":"python"}
```

Likes : une réaction « j'aime » relie un utilisateur à un post avec un likedAt. Par conception, **un utilisateur ne peut liker un post qu'une seule fois**, et la probabilité d'aimer est plus forte à l'intérieur d'une même communauté (biais).

```
{"userId":"u_02000","postId":"p_000123","likedAt":"2025-04-27T08:15:32"}
```

Commentaires (Comment) : les commentaires sont des objets à part entière avec un id, un authorId, un postId, un content et un createdAt. Cette séparation permet d'étudier les fils de discussion et leurs rythmes d'apparition indépendamment du post d'origine.

```
{"id":"c_0001023","authorId":"u_00567","postId":"p_000123","createdAt":"2025-06-02T19:41:10","content":"Texte aléatoire de X mots"}
```

Groupes (Group) : ils portent id, name, description, visibility et createdAt. Les **adhésions** associent un userId à un groupId avec un role (admin, moderator, member) et un joinedAt. Les groupes exploitent la proximité communautaire pour simuler des espaces d'intérêt cohérents.

```
{"id":"g_012","name":"Tech - Plateforme durable","visibility":"public","createdBy":"u_00077","description":"...", "createdAt":"2025-05-12T10:02:00"}
{"userId":"u_00077","groupId":"g_012","role":"admin","joinedAt":"2025-05-12T10:02:00"}
```

Signalements (Report) : un **report** contient id, reason (spam, hate_speech, harassment, fake_news, inappropriate_content), un status (open, in_review, resolved, dismissed) et createdAt. Un second fichier relie chaque report à son **auteur** et à sa **cible** (Post, Comment ou User).

```
{"id":"r_00042","reason":"spam","status":"in_review","createdAt":"2025-07-01T13:33:21"}
{"reportId":"r_00042","reportedBy":"u_01010","targetType":"Post","ta"}
```

```
rgetId":"p_000123"}
```

Ensemble, ces jeux couvrent **neuf dimensions** : `users.ndjson`, `follows.ndjson`, `posts.ndjson`, `post_tags.ndjson`, `likes.ndjson`, `comments.ndjson`, `groups.ndjson`, `group_members.ndjson`, `reports.ndjson`, `report_relations.ndjson`. Les distributions (visibilités, privacy, homophilie, popularité) sont contrôlées pour permettre des analyses causales et des démonstrations produit.

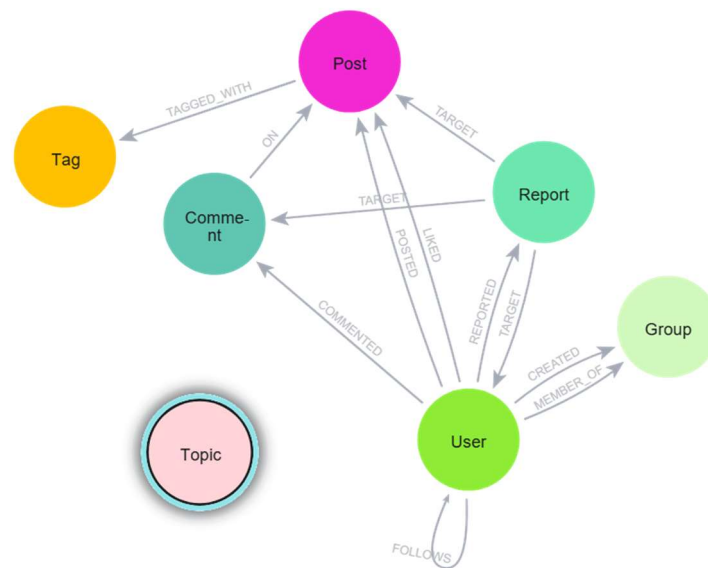
2) Justification du modèle de données choisi

Le **modèle graphe** (Neo4j - property-graph) a été retenu parce que la valeur métier provient des relations, qui suit qui, qui interagit avec quoi, où émergent les communautés et comment se propagent les contenus. Les **nœuds** (User, Post, Comment, Group, Report, Tag) et **relations** (FOLLOWS, POSTED, LIKED, COMMENTED, ON, MEMBER_OF, CREATED, REPORTED, TARGET, correspondent exactement aux questions que l'on pose :

- Fil d'actualité : `(:User)-[:FOLLOWS]->(:User)-[:POSTED]->(:Post)`
- Recommandation « à suivre » : chemins de longueur 2–3 et **comptage de chemins**.
- Tendances / thèmes : remontées via `(:Post)-[:...]->(:Tag)` et agrégations temporelles sur les propriétés de relations (`LIKED.likedAt`).
- Communautés : composantes denses obtenues par structure du graphe (densité intra-communautés déjà injectée à la génération).
- Gouvernance & éthique : explicabilité des suggestions (chemins observables), filtrage par **privacy/visibility** explicite et **modération**.

Ce choix évite des jointures coûteuses et rend les **parcours** lisibles avec **Cypher**. Les médias (images/vidéos) ne sont pas stockés en base, seule l'URL est conservée, ce qui respecte la sobriété et simplifie la montée en charge (le stockage objet/CDN est conçu pour ça). Enfin, certaines propriétés (compteurs de likes, de commentaires) sont matérialisées dans les nœuds Post pour permettre des lectures rapides.

3) Aperçu du schéma et structure JSON



{{identity:-

555,labels:["Comment"],properties:{name:"Comment",indexes:[],constraints:["Constraint(id=7, name='comment_id', type='NODE PROPERTY UNIQUENESS', schema=(:Comment {id}), ownedIndex=6)"]},elementId:"-555"},{identity:-

559,labels:["Group"],properties:{name:"Group",indexes:["visibility"],constraints:["Constraint(id=15, name='group_id', type='NODE PROPERTY UNIQUENESS', schema=(:Group {id}), ownedIndex=14)"]},elementId:"-559"},{identity:-

553,labels:["User"],properties:{name:"User",indexes:["username","privacy"],constraints:["Constraint(id=3, name='user_id', type='NODE PROPERTY UNIQUENESS', schema=(:User {id}), ownedIndex=2)","Constraint(id=21, name='user_email', type='NODE PROPERTY UNIQUENESS', schema=(:User {email}), ownedIndex=20)"]},elementId:"-553"},{identity:-

558,labels:["Report"],properties:{name:"Report",indexes:[],constraints:["Constraint(id=13, name='report_id', type='NODE PROPERTY UNIQUENESS', schema=(:Report {id}), ownedIndex=12)"]},elementId:"-558"},{identity:-

554,labels:["Post"],properties:{name:"Post",indexes:["visibility"],constraints:["Constraint(id=5, name='post_id', type='NODE PROPERTY UNIQUENESS', schema=(:Post {id}), ownedIndex=4)"]},elementId:"-554"},{identity:-

556,labels:["Tag"],properties:{name:"Tag",indexes:[],constraints:["Constraint(id=9, name='tag_name', type='NODE PROPERTY UNIQUENESS', schema=(:Tag {name}), ownedIndex=8)"]},elementId:"-556"},{identity:-

557,labels:["Topic"],properties:{name:"Topic",indexes:[],constraints:["Constraint(id=11, name='topic_id', type='NODE PROPERTY UNIQUENESS', schema=(:Topic {id}), ownedIndex=10)"]},elementId:"-557"}]

4) Procédure de génération

La génération s'appuie sur `generator.py` et repose sur des **processus probabilistes contrôlés** pour concilier réalisme et reproductibilité.

Le script fixe des graines (`Faker.seed(42)`, `random.seed(42)`) pour des données stables entre deux exécutions. Les **volumes cibles** sont paramétrables en tête de fichier : `N_USERS=3000`, `TARGET_POSTS=5000`, `TARGET_LIKES=15000`, `TARGET_COMMENTS=8000`, `N_GROUPS=50`, `N_REPORTS=200`, sur `DAYS=365`. Les distributions sont explicites : `PRIVACY` (55 % private, 45 % public) et visibilité **des posts** (60 % public, 30 % `followers_only`, 10 % private).

Chaque utilisateur est assigné de manière équilibrée à l'une des cinq **communautés** thématiques, ce qui influence ensuite ses liens, ses thèmes de posts et ses tags.

Chaque utilisateur est également généré avec une **adresse email unique** et un **mot de passe haché**. Les emails sont créés via la librairie `Faker` pour garantir des valeurs réalistes, tandis que les mots de passe sont hachés afin de simuler un système d'authentification sécurisé même dans un environnement de test.

Construction du graphe social :

1. **Intra-communautés** : pour chaque communauté, chaque membre suit un petit échantillon de pairs (jusqu'à 15) pour constituer des noyaux.
2. **Inter-communautés** : des follows aléatoires relient les clusters pour éviter un graphe disjoint.
3. **Attachement préférentiel** : on renforce la probabilité de suivre des comptes déjà très suivis (score basé sur l'in-degree), afin de faire émerger des **hubs**. Les **doublons** et **auto-follows** sont empêchés.

Le nombre de posts par auteur est proportionnel à $1 + \sqrt{\text{in_degree}}$, cette loi privilégie légèrement les comptes populaires. Chaque post reçoit une date sur la fenêtre d'un an, une **visibility**, une **mediaUrl** pour 70 % des posts et un **topic** dérivé de la communauté de l'auteur.

Environ 70 % des posts reçoivent entre **1 et 5 tags**. Pour chacun, il y a 80 % de chance de piocher dans le lexique de la communauté, sinon dans un lexique générique. On évite les doublons intra-post pour que les comptages restent propres.

Pour chaque like à générer, on choisit un post puis on échantillonne un utilisateur, si l'utilisateur appartient à la même communauté que l'auteur du post, la probabilité d'aimer est 70%, sinon 30%. Un set (`user`, `post`) garantit l'unicité et les **auto-likes** sont exclus.

On choisit un post, on tire un auteur (avec une légère aversion aux auto-commentaires), on des mots en français et on horodate. Les commentaires sont des nœuds distincts pour permettre des analyses fines.

Pour chaque groupe, on choisit un créateur dans la communauté cible, on attribue un nom et une description, on tire une *visibility*, puis on échantillonne **5 à 30 membres** de la même communauté. Le créateur devient l'**admin**, environ 10 % des autres membres deviennent **moderators**, le reste **members**. Les dates d'adhésion correspondent à la date de création du groupe pour un scénario simple.

On génère des **reports** indépendants, avec une raison, un statut et une date. On relie ensuite $(:User) - [:REPORTED] \rightarrow (:Report) - [:TARGET] \rightarrow (Post | Comment | User)$ en veillant à éviter l'auto-signallement sur la cible user. Les cibles sont tirées parmi tous les posts, tous les commentaires et un échantillon d'utilisateurs, ce qui fournit de la variété pour tester les modérations.

Après la génération des likes et des commentaires, on agrège par `postId` pour renseigner `likeCount` et `commentCount` sur chaque post.

Chaque collection est sérialisée en **NDJSON** dans le dossier `data`. Ces fichiers sont ensuite ingérés dans Neo4j par `import.py`, que nous allons expliquer dans la partie suivante.

5. Scénarios de test réalisés

1) Insertion de données

Comme décrit dans la partie précédente, nous générons nos données avec un script Python, dans cette partie, nous allons expliquer leurs insertions. Le script d'import va utiliser le package « dotenv » pour récupérer les informations d'identification contenue dans le fichier d'environnement (non commit). Pour des raisons de simplicités, nous avons créé une classe qui contient toutes les fonctions, voici la liste :

- `__init__(self, uri, user, password)` : se connecte à la base
- `close(self)` : ferme la connexion à la base
- `clear_database(self)` : nettoie la base
- `create_constraints_indexes(self)` : crée les contraintes et les index
- `load_ndjson(self, filename)` : charge un fichier NDJSON
- `import_users(self)` : importe les utilisateurs avec toutes leurs propriétés, y compris email et password (haché).
- `import_follows(self)` : import des relations d'abonnements
- `import_posts(self)` : import des publications
- `import_post_tags(self)` : import des tags
- `import_likes(self)` : import des likes
- `import_comments(self)` : import des commentaires
- `import_groups(self)` : import des groupes
- `import_group_members(self)` : import des membres des groupes
- `import_reports(self)` : import des signalements
- `import_report_relations(self)` : import des relations de signalements
- `run_full_import(self)` : lance l'import complet

Une fois que ce script est lancé, les données conservées dans le dossier data seront insérées dans notre graphe Neo4j.

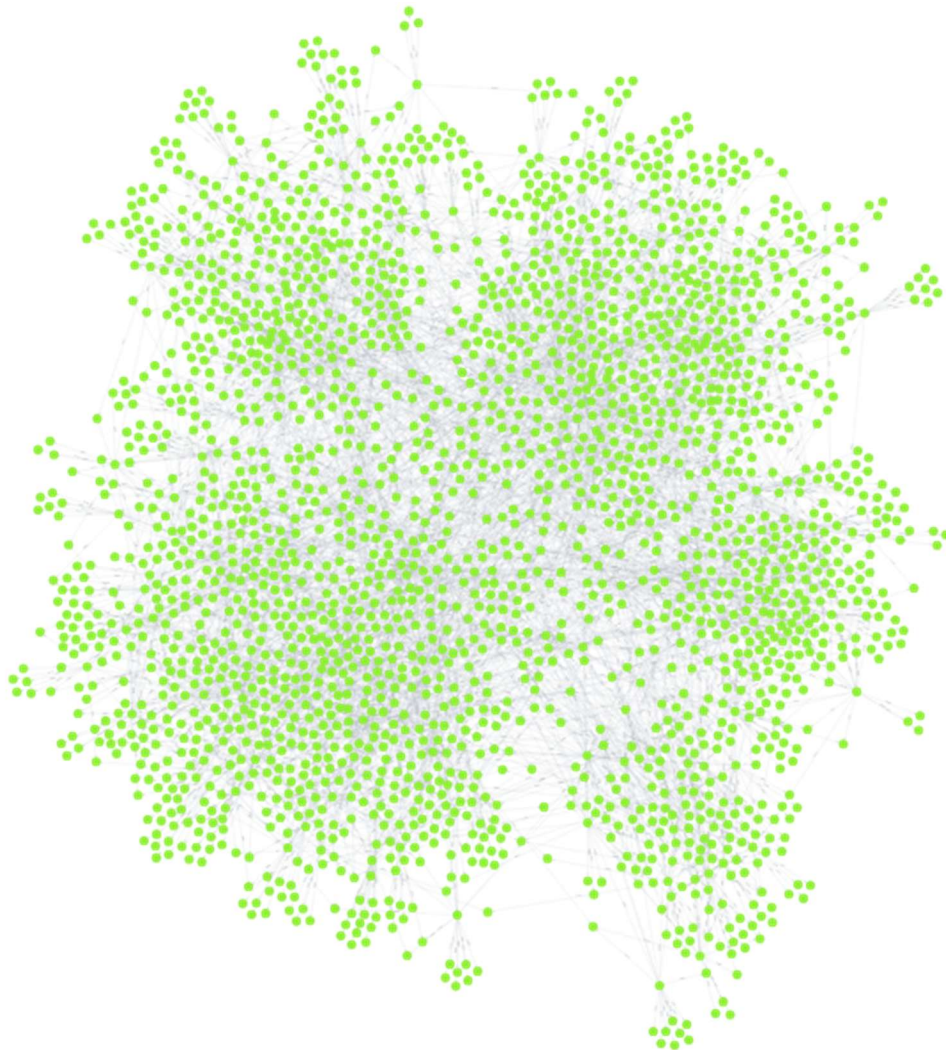
2) Requêtes simples et avancées

Vous pourrez regarder dans cette partie différentes requête pour comprendre l'étendue de notre modèle :

- Voir le réseau de “follows” entre utilisateurs :

```
MATCH (a:User)-[f:FOLLOWS]->(b:User)
```

```
RETURN a, f, b
```



- Voir les signalements (reports) :

```
MATCH (r:Report)-[t]->(n)
```

```
RETURN r, t, n
```

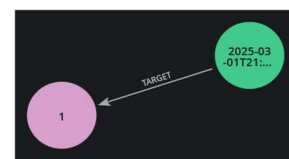
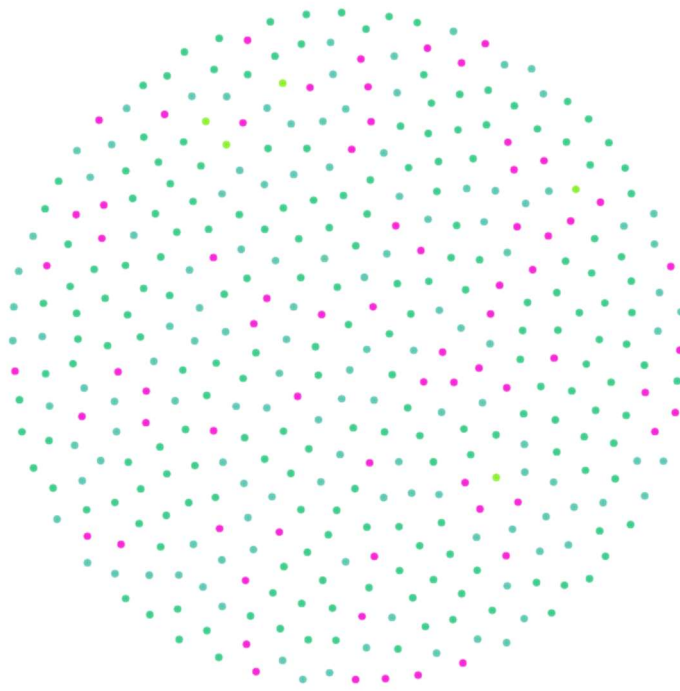


Figure 9 : un report et son lien

- Identifier les 10 tags les plus populaires (basé sur le nombre de posts) :

```
MATCH (p:Post)-[TAGGED_WITH]->(t:Tag)
RETURN t.name AS tag, count(p) AS postCount
ORDER BY postCount DESC
LIMIT 10
```

"kubernetes"	465
"api"	461
"javascript"	459
"cloud"	448
"nodejs"	448
"python"	446
"ia"	444
"react"	440
"dev"	426
"docker"	426

3) Agrégations

Dans cette partie nous allons montrer des vues « macro » sur l'activité et la structure du modèle. Ci-dessous, des exemples classés par thème avec leurs résultats :

- **Nombre de nœuds par type :**

```
MATCH (n)

RETURN labels(n) AS nodeType, count(n) AS nb_noeuds

ORDER BY nbNoeuds DESC;
```

	typeNoeuds	nbNoeuds
1	["Comment"]	8000
2	["Post"]	5000
3	["User"]	3000
4	["Report"]	200
5	["Tag"]	60
6	["Group"]	50

- **Nombre de relations par type :**

```
MATCH ()-[r]->()

RETURN type(r) AS noeuds, count(*) AS nbRelations

ORDER BY nbRelations DESC;
```

	noeuds	nbRelations
1	"FOLLOWS"	61887
2	"LIKED"	15000
3	"TAGGED_WITH"	9774
4	"COMMENTED"	8000
5	"ON"	8000
6	"POSTED"	5000
7	"MEMBER_OF"	956
8	"REPORTED"	200
9	"TARGET"	200
10	"CREATED"	50

- Répartition des utilisateurs par communauté :

```
MATCH (u:User)

RETURN u.community AS community, count(*) AS users

ORDER BY users DESC;
```

	visibility	posts
1	"public"	3014
2	"followers_only"	1488
3	"private"	498

- Statistiques (moyenne, médiane, percentiles)

```
MATCH (:User)-[f:FOLLOWS]->(u:User) WITH u, count(f) AS followers

RETURN round(avg(followers),2) AS avg_followers,

       percentileCont(followers,0.5) AS Median,

       percentileCont(followers,0.9) AS TOP10;
```

	avg_followers	Median	TOP10
1	20.63	17.0	32.0

- Engagement 7 derniers jours :

```
MATCH (p:Post)

WHERE p.createdAt >= datetime() - duration('P7D')

OPTIONAL MATCH (p)-[l:LIKED]-()

OPTIONAL MATCH (p)-[c:ON]-(c:Comment)

RETURN count(DISTINCT p) AS posts_7d, count(DISTINCT l) AS likes_7d, count(DISTINCT c) AS comments_7d
```

	posts_7d	likes_7d	comments_7d
1	81	252	107

4) Mise à jour

Voici quelques requêtes qui vont illustrer comment mettre à jour des données dans une base Neo4j :

- Changer la confidentialité d'un utilisateur :

```
MATCH (u:User {id:'u_00042'})
SET u.privacy = 'private',
RETURN u.id, u.privacy, u.updatedAt;
```

- Résoudre un report :

```
MATCH (r:Report {id:'r_000077'})
SET r.status = 'resolved'
RETURN r.id, r.status, r.resolvedAt;
```

Report	
Key	Value
createdAt	2025-06-08T14:25:03Z
reason	"inappropriate_content"
id	"r_00007"
status	"open"

Figure 10 : avant la modification

Report	
Key	Value
createdAt	2025-06-08T14:25:03Z
reason	"inappropriate_content"
id	"r_00007"
status	"resolved"

Figure 11 : après la modification

- Mise à jour en lot, basculer en « private » les posts de plus de d'un an

```
MATCH (p:Post) WHERE p.createdAt < datetime() - duration('P365D')
SET p.visibility = 'private'
RETURN count(p) AS postsUpdated;
```

postsUpdated

118

5) Suppression

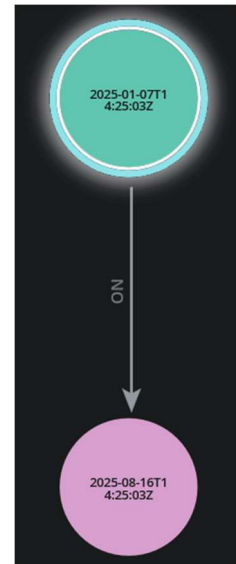
Voici quelques requêtes qui vont illustrer comment supprimer des données dans une base Neo4j :

- Supprimer un commentaire et ses relations :

```
// Pour retrouver les relations
MATCH (c:Comment {id:'c_0000001'})
OPTIONAL MATCH (c)-[r]->(n)
RETURN c, collect(r) AS relations, collect(n) AS relie_a
// Pour effacer le commentaire et ses relations
MATCH (c:Comment {id:'c_0000001'})
DETACH DELETE c;
```

- Nettoyage en lot : supprimer les reports « dismissed » de plus d'un an :

```
MATCH (r:Report {status:'dismissed'})
WHERE exists(r.createdAt) AND r.createdAt < datetime() - duration('P365D')
DETACH DELETE r;
```

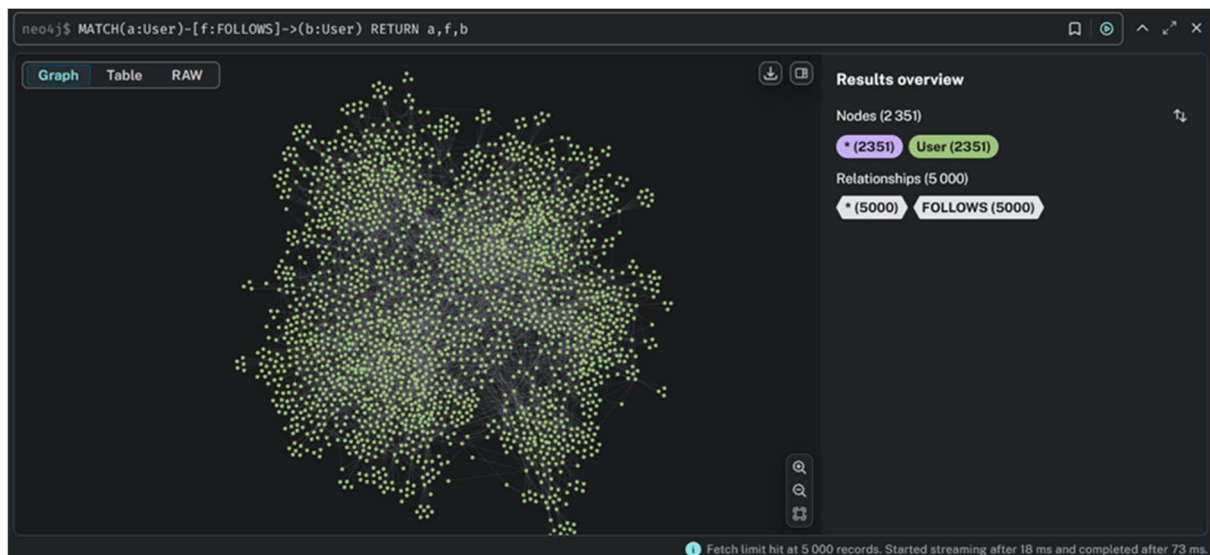


6) Analyse critique

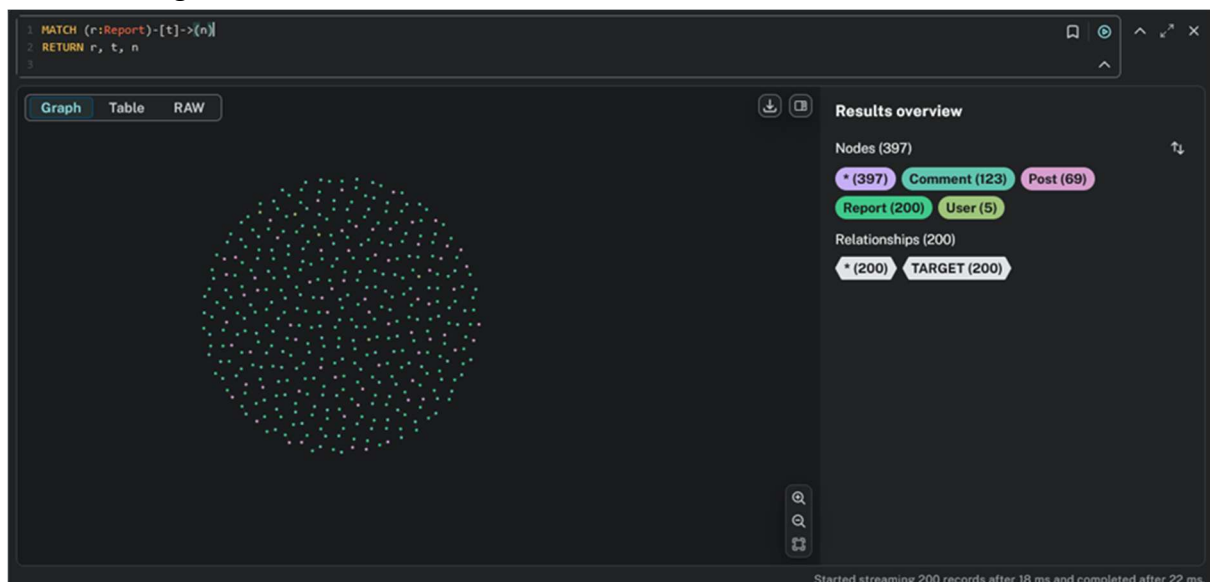
L'analyse des différents scénarios de test réalisés sur notre base Neo4j (insertions, requêtes simples et avancées, agrégations, mises à jour et suppressions) met en évidence à la fois la robustesse de notre modèle, mais aussi certaines limites techniques rencontrées.

Temps de réponse :

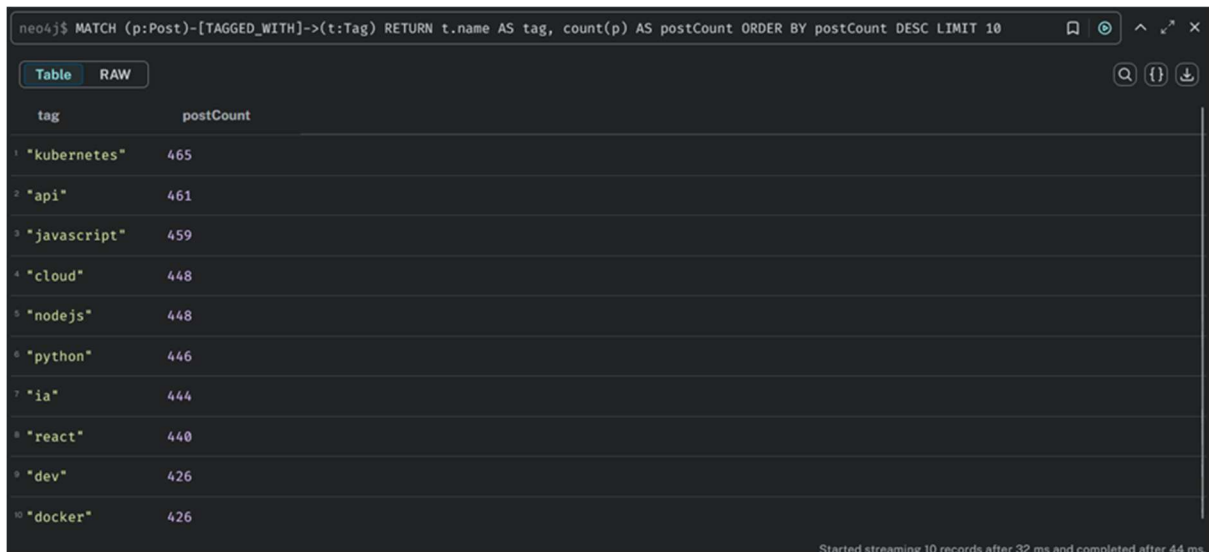
- Les relations des utilisateurs entre eux : 73ms



- Les signalements et leurs relations : 22ms



- Identifier les 10 tags les plus populaires (basé sur le nombre de posts) : 44ms



```
neo4j$ MATCH (p:Post)-[TAGGED_WITH]->(t:Tag) RETURN t.name AS tag, count(p) AS postCount ORDER BY postCount DESC LIMIT 10
```

	tag	postCount
1	"kubernetes"	465
2	"api"	461
3	"javascript"	459
4	"cloud"	448
5	"nodejs"	448
6	"python"	446
7	"ia"	444
8	"react"	440
9	"dev"	426
10	"docker"	426

Started streaming 10 records after 32 ms and completed after 44 ms.

- Nombre de relations par type : 132ms



```
neo4j$ MATCH ()-[r]->() RETURN type(r) AS noeuds, count(*) AS nbRelations ORDER BY nbRelations DESC
```

	noeuds	nbRelations
1	"FOLLOWS"	61887
2	"LIKED"	15000
3	"TAGGED_WITH"	9774
4	"COMMENTED"	8000
5	"ON"	8000
6	"POSTED"	5000
7	"MEMBER_OF"	956
8	"REPORTED"	200
9	"TARGET"	200
10	"CREATED"	50

Started streaming 10 records after 1 ms and completed after 132 ms.

On peut noter que nos résultats restent globalement cohérents avec les requêtes demandées. Néanmoins, le stockage des médias reste problématique, pouvoir stocker les médias en soi et non des URLs, seraient sans doute intéressant pour vendre ces données (avec l'accord de l'utilisateur) à des modèles d'IA dans le cadre de leurs entraînements. L'une des solutions pourrait être de stocker les médias dans une base MongoDB sous un format blob, et nos deux bases seront reliés à l'aide de DBeaver.

6. Analyse critique et retour d'expérience

1) Points forts et points faible de Neo4j

L'expérimentation de Neo4j nous a permis d'observer concrètement les avantages et les limites d'une base de données orientée graphe dans un contexte de réseau social.

Points forts :

Neo4j s'est révélé particulièrement adapté à la modélisation de données sociales. Le langage Cypher permet de représenter de manière intuitive les relations entre utilisateurs, publications, commentaires et interactions. L'outil offre également une visualisation claire du graphe, facilitant la compréhension du modèle et la navigation dans les données. De plus, les performances observées sur des requêtes confirment la pertinence du choix d'une base orientée graphe pour ce type d'application.

Points faibles :

Certaines limites ont été constatées, l'insertion de grands volumes de données peut devenir lente sans optimisation préalable (indexation, contraintes uniques). La gestion de la mémoire est sensible, en particulier lors de la manipulation de milliers de relations et de nœuds. Par ailleurs, les agrégations statistiques complexes sont parfois moins naturelles à exprimer en Cypher qu'en MongoDB ou en SQL « classique », et le stockage de contenus multimédias nécessite une solution externe, Neo4j n'étant pas conçu pour gérer directement des fichiers volumineux.

2) Difficultés et amélioration possible du projet

Au cours du projet, nous avons été confrontés à plusieurs difficultés. La maîtrise du Cypher et de la modélisation graphe a nécessité un temps d'adaptation, en particulier pour formuler des requêtes complexes et naviguer dans les relations multi-niveaux.

Améliorations possibles :

Pour améliorer le projet, plusieurs pistes peuvent être envisagées. L'optimisation des performances pourrait passer par l'utilisation de batchs pour l'insertion massive de données, plus d'index et des contraintes supplémentaires, ainsi que l'exploitation du [plugin APOC](#) pour simplifier certaines opérations complexes. Enfin, on aurait pu trouver une différente solution pour le stockage des médias, l'hypothèse principale était de connecter la base Neo4j avec une base MongoDB, où les documents sont beaucoup plus adaptés pour gérer ce genre de données.

3) Répartition réelle du travail

Valentin PERROUX	Sefer TASDEMIR	Feyza TINASTEPE
• Étude avant-projet : analyse et comparaison • Description du jeu de données créé	• Présentation du groupe et du contexte choisi • Expérimentation de Neo4j • Scénarios de test réalisés • Création du modèle • Génération/insertion des données	• Présentation du groupe et du contexte choisi • Retour d'expérience • Requêtes simples et avancées • Présentation PowerPoint